Software Requirements Specification

1. Introduction

1.1 Purpose: This document specifies the functional and non-functional requirements for the Booking Platform REST API, a backend system allowing authenticated users to manage bookable services and customers to create bookings against those services.

1.2 Scope: The system exposes a versioned REST API covering user authentication, service management, and booking management. No frontend UI is included; the API is consumed via HTTP clients (Swagger, Postman, or any REST client).

1.3 Technology Stack: NestJS (TypeScript), PostgreSQL, Prisma ORM, JWT authentication, Swagger/OpenAPI documentation.

2. Overall Description

2.1 User Classes: Two actors interact with the system: (a) **Registered Users** — authenticated staff/owners who manage services and oversee bookings; (b) **Customers** — unauthenticated members of the public who create bookings.

2.2 Operating Environment: Node.js 20+ runtime, PostgreSQL 16 database, deployable via Docker or any Node-compatible host.

3. Functional Requirements

FR-1 Authentication

- Users can register with email, password and name (password hashed with bcrypt).
- Users can log in and receive a short-lived access token and a longer-lived refresh token (JWT).
- Users can exchange a valid refresh token for a new token pair.

FR-2 Service Management (authenticated users only)

- Create, update, delete a service; list all services; retrieve a single service by ID.
- Service fields: title, description, duration (minutes), price, isActive.
- List endpoint supports pagination, text search, and active-status filtering.

FR-3 Booking Management

- Customers create a booking without authentication (public endpoint).
- Authenticated users can list, retrieve, update the status of, and cancel bookings.
- Booking fields: customerName, customerEmail, customerPhone, serviceId, bookingDate, bookingTime, status, notes.
- Booking status is one of: PENDING, CONFIRMED, CANCELLED, COMPLETED.

4. Business Rules

- A booking must reference an existing, active service.
- Booking date cannot be in the past.
- A cancelled booking can never transition to any other status (including COMPLETED).

- A completed booking cannot be cancelled.
- Duplicate bookings for the same service + date + time are rejected (enforced at the database level).

5. Non-Functional Requirements

Category	Requirement
Security	Passwords hashed with bcrypt; JWT access/refresh tokens with separate secrets; Helmet security headers; global rate l
Validation	All input validated via DTOs (class-validator); unknown fields rejected.
Error Handling	All errors normalized to a single JSON shape; no internal stack traces exposed.
Consistency	All successful responses use a single { success, message, data } envelope.
Maintainability	Modular NestJS architecture; controllers contain no business logic.
Documentation	Swagger/OpenAPI UI and a Postman collection are provided.
Performance	Paginated list endpoints to avoid unbounded result sets.

6. Data Model (Summary)

Entity	Key Fields	Relationships
User	id, email, password (hashed), name	One user owns many Services
Service	id, title, description, duration, price, isActive, ownerId	Belongs to one User; has many Bookings
Booking	id, customerName, customerEmail, customerPhone, serviceId, booki	ngDate, boBelongs to one ServicekingTime, status, notes

7. API Surface (Summary)

Method	Endpoint	Auth
POST	/auth/register, /auth/login, /auth/refresh	Public / Refresh token
POST /GET /PATCH /DELETE	/services, /services/:id	Required
POST	/bookings	Public
GET /PATCH	/bookings, /bookings/:id, /bookings/:id/status, /bookings/:id/cancel	Required
GET	/health	Public

8. Assumptions & Constraints

- Only the owning user may update or delete a given service (stricter reading of "authenticated users manage services").
- Refresh tokens are stateless JWTs and are not individually revocable before expiry.
- No UI/frontend is in scope; the system is API-only.